

Mallonee_Tilled_ERW_Analysis_v3

May 22, 2026

1 Mallonee Family Farm — Tilled Field ERW Project

1.1 Baseline Characterization & Pre/Post Application Analysis

Project: Enhanced Rock Weathering (ERW) — Tilled Stratum

Methodology: Rainbow Standard Method 2 (Solid Phase Mass Balance)

Feedstock: TS-1 Olivine (47.3 wt% MgO), applied at 7.4 t/ha

Lab Reports: Baseline — ALS LR25329328 | Post-Application — ALS RE25329687

Prepared by: Basic Dust SPC Inc / Goal300 Heather Langsner, MS, ChemE'27 Columbia University

1.1.1 Notebook Structure

1. **Setup & Data Loading** — imports, file paths, data ingestion
2. **LOI Normalization** — correcting for organic matter dilution effects
3. **Baseline Characterization** — pH, bulk density, and geochemistry by group
4. **Control Plot Representativeness** — SMD check (Rainbow requirement: $SMD < 10\%$)
5. **Pre vs Post Application Analysis** — paired T-tests on MgO and base cation pool
6. **Summary & Audit Table** — exportable results for submission

Note on LOI Normalization: Organic matter content (reflected in Loss on Ignition, LOI) at this site is high and spatially variable (15–19%). Raw oxide concentrations are therefore subject to dilution effects in high-LOI samples. All elemental concentrations are normalized to the mineral fraction using: $\text{normalized\%} = (\text{raw\%} / (\text{Total\%} - \text{LOI\%})) \times 100$. This expresses concentrations per unit mineral mass, enabling direct comparison across zones. Also please note that a change in file naming conventions between pre and post by ALS created the need to set up pairing keys as you'll see below. Post-application pH, bulk density, and CEC measurements are pending as of peer review but may arrive in coming days or weeks. These measurements are planned for the next monitoring round (target: September 2026 final sampling). Archived samples from the post-application round are available for retrospective analysis. CDR quantification for this period relies on solid-phase MgO mass balance per Rainbow Method 2, which does not require post-application pH or BD as primary inputs. Also please note that rather than incurring further expense to conduct control sampling and lab analysis for post application, we decided to use the data from the control zone in the pre-application data.

1.2 1. Setup & Data Loading

```
[2]: import pandas as pd
import numpy as np
import re
from scipy import stats
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import warnings
warnings.filterwarnings('ignore')

# File paths
PATH_PRE = 'csv files/Baseline Mineral analysis from ALS - LR25329328 fixed.
↳ csv'
PATH_POST = 'csv files/Tilled - Mallonee post-deployment ALS mineral analysis
↳ RE25329687.csv'
PATH_PH = 'csv files/ph_df_Mallonee_baseline.csv'
PATH_RAND = 'csv files/Random Interior Samples tilled pre-treatment and control
↳ Mallonee - Sheet1 (1).csv'

OXIDE_COLS =
↳ ['SiO2', 'Al2O3', 'Fe2O3', 'CaO', 'MgO', 'Na2O', 'K2O', 'Cr2O3', 'TiO2', 'MnO', 'P2O5']
BASE_OXIDES = ['CaO', 'MgO', 'Na2O', 'K2O']
MW = {'CaO': 56.08, 'MgO': 40.30, 'Na2O': 61.98, 'K2O': 94.20}
CHARGE = {'CaO': 2, 'MgO': 2, 'Na2O': 2, 'K2O': 2}

print('Libraries loaded')
import os
os.makedirs('outputs', exist_ok=True)
print('outputs folder ready')
```

Libraries loaded
outputs folder ready

```
[3]: # ALS CSV loader
def load_als_csv(path):
    raw = pd.read_csv(path, header=None)
    header_row = raw.index[raw[0].astype(str).str.strip().eq('SAMPLE')][0]
    df = raw.iloc[header_row:].reset_index(drop=True)
    df.columns = df.iloc[0].astype(str).str.strip()
    df = df.iloc[2:].reset_index(drop=True)
    df = df.apply(lambda c: c.str.strip() if c.dtype == 'object' else c)
    return df

# LOI normalizer
def loi_normalize(df, oxide_cols):
    df = df.copy()
```

```

for col in oxide_cols + ['LOI', 'Total']:
    df[col] = pd.to_numeric(df[col], errors='coerce')
anhydrous = df['Total'] - df['LOI']
for col in oxide_cols:
    df[col + '_norm'] = (df[col] / anhydrous) * 100
return df

# Pair key functions
def pair_key_pre(sample):
    s = str(sample).strip()
    # Treatment interior random samples -> R01-R07
    m = re.fullmatch(r'Treatment_(\d+)', s, re.IGNORECASE)
    if m: return f"R{int(m.group(1)):02d}"
    # Treatment paired zone samples -> Z01-Z10
    m = re.fullmatch(r'Treatment_z(\d+)', s, re.IGNORECASE)
    if m: return f"Z{int(m.group(1)):02d}"
    # Control paired zone samples -> CZ01-CZ10
    m = re.fullmatch(r'Control_z(\d+)', s, re.IGNORECASE)
    if m: return f"CZ{int(m.group(1)):02d}"
    # Control interior samples -> CR01-CR05
    m = re.fullmatch(r'Control(\d+)', s, re.IGNORECASE)
    if m: return f"CR{int(m.group(1)):02d}"
    return None

def pair_key_post(sample):
    s = str(sample).strip()
    m = re.fullmatch(r'TT\s*rand\s*(\d+)', s, re.IGNORECASE)
    if m: return f"R{int(m.group(1)):02d}"
    m = re.fullmatch(r'TZ\s*(\d+)', s, re.IGNORECASE)
    if m: return f"Z{int(m.group(1)):02d}"
    return None

def mmol_charge(row, suffix):
    total = 0.0
    for ox in BASE_OXIDES:
        val = pd.to_numeric(row.get(f'{ox}_{suffix}', np.nan), errors='coerce')
        if pd.isna(val): continue
        total += (val * 10 / MW[ox]) * 1000 * CHARGE[ox]
    return total

print('Helper functions defined')

```

Helper functions defined

```

[4]: # Load & normalize baseline
dfpre = load_als_csv(PATH_PRE)
dfpre = loi_normalize(dfpre, OXIDE_COLS)

```

```

dfpre['pair_key'] = dfpre['SAMPLE'].apply(pair_key_pre)

# Load & normalize post-application
dfpost = load_als_csv(PATH_POST)
dfpost = loi_normalize(dfpost, OXIDE_COLS)
dfpost['pair_key'] = dfpost['SAMPLE'].apply(pair_key_post)

# Load pH & bulk density data
dfpH = pd.read_csv(PATH_PH)

# Load random interior BD/pH samples
dfRand = pd.read_csv(PATH_RAND, skiprows=1)
dfRand.columns = [c.strip() for c in dfRand.columns]
dfRand = dfRand.dropna(axis=1, how='all')

print(f'Baseline rows loaded:           {len(dfpre)}')
print(f'Post-application rows loaded:  {len(dfpost)}')
print(f'pH/BD rows loaded:                {len(dfpH)}')
print(f'Random interior BD/pH rows:       {len(dfRand)}')
print()
print('Baseline sample IDs:')
print(dfpre[['SAMPLE', 'pair_key']].to_string(index=False))

```

```

Baseline rows loaded:           32
Post-application rows loaded:  17
pH/BD rows loaded:             20
Random interior BD/pH rows:    12

```

Baseline sample IDs:

SAMPLE	pair_key
Control_z8	CZ08
Control_z3	CZ03
Control_z2	CZ02
Control_z1	CZ01
Control_z10	CZ10
Control_z4	CZ04
Control_z5	CZ05
Control_z6	CZ06
Control_z7	CZ07
Control_z9	CZ09
Control1	CR01
Control2	CR02
Control3	CR03
Control4	CR04
Control5	CR05
Treatment_1	R01
Treatment_2	R02

Treatment_3	R03
Treatment_4	R04
Treatment_5	R05
Treatment_6	R06
Treatment_7	R07
Treatment_z1	Z01
Treatment_z10	Z10
Treatment_z2	Z02
Treatment_z3	Z03
Treatment_z4	Z04
Treatment_z5	Z05
Treatment_z6	Z06
Treatment_z7	Z07
Treatment_z8	Z08
Treatment_z9	Z09

```
[5]: # Split baseline into four sampling groups
treat_paired = dfpre[dfpre['SAMPLE'].str.match(r'Treatment_z\d+',
↪case=False, na=False)].copy()
treat_interior = dfpre[dfpre['SAMPLE'].str.match(r'Treatment_\d+$',
↪case=False, na=False)].copy()
ctrl_paired = dfpre[dfpre['SAMPLE'].str.match(r'Control_z\d+',
↪case=False, na=False)].copy()
ctrl_interior = dfpre[dfpre['SAMPLE'].str.match(r'Control\d+$',
↪case=False, na=False)].copy()

GROUPS = [
    ('Treatment Paired', treat_paired),
    ('Treatment Interior', treat_interior),
    ('Control Paired', ctrl_paired),
    ('Control Interior', ctrl_interior),
]

print('Group sizes:')
for name, df in GROUPS:
    print(f' {name:<22} n={len(df)}')
```

Group sizes:

Treatment Paired	n=10
Treatment Interior	n=7
Control Paired	n=10
Control Interior	n=5

1.3 2. Baseline Characterization

1.3.1 2a. Bulk Density & pH

Field measurements taken prior to olivine application across the 10 paired zones and 12 randomly distributed interior samples.

```
[6]: # BD/pH stats - paired zones
for label, mask in [
    ('Treatment Paired Zones', dfpH['Treatment'] == 'Baseline_
    ↪(Pre-Application)'),
    ('Control Zones',          dfpH['Treatment'] == 'Baseline Control'),
]:
    sub = dfpH[mask]
    if len(sub) == 0:
        print(f'\n{label}: no rows found')
        continue
    print(f'\n{"="*50}')
    print(f'{label} (n={len(sub)})')
    print(f'{"="*50}')
    for col in ['BD', 'pH lab', 'pH sentron', 'Moisture content']:
        if col in sub.columns:
            s = sub[col]
            print(f'    {col:<20} mean={s.mean():.3f} std={s.std():.3f} min={s.
            ↪min():.3f} max={s.max():.3f}')

# BD/pH stats - random interior samples
print(f'\n{"="*50}')
print('Random Interior Samples')
print(f'{"="*50}')
for trt in ['control', 'pre-olivine']:
    sub = dfRand[dfRand['Treatment'] == trt]
    label = 'Control Interior' if trt == 'control' else 'Treatment Interior'
    print(f'\n {label} (n={len(sub)})')
    for col in ['BD', 'pH lab', 'Moisture content']:
        if col in sub.columns:
            s = pd.to_numeric(sub[col], errors='coerce')
            print(f'    {col:<20} mean={s.mean():.3f} std={s.std():.3f} 
            ↪min={s.min():.3f} max={s.max():.3f}')
```

=====

Treatment Paired Zones (n=10)

=====

BD	mean=0.789	std=0.071	min=0.688	max=0.902
pH lab	mean=6.107	std=0.321	min=5.660	max=6.780
pH sentron	mean=5.801	std=0.864	min=5.100	max=7.100
Moisture content	mean=0.358	std=0.065	min=0.271	max=0.469

```

=====
Control Zones (n=10)
=====
BD                mean=0.785  std=0.099  min=0.580  max=0.958
pH lab            mean=6.033  std=0.239  min=5.660  max=6.440
pH sentron        mean=5.798  std=0.759  min=4.870  max=7.030
Moisture content  mean=0.357  std=0.078  min=0.269  max=0.521

```

```

=====
Random Interior Samples
=====

```

```

Control Interior (n=5)
BD                mean=0.819  std=0.149  min=0.718  max=1.070
pH lab            mean=6.178  std=0.284  min=5.700  max=6.420
Moisture content  mean=0.122  std=0.018  min=0.106  max=0.151

```

```

Treatment Interior (n=7)
BD                mean=1.132  std=0.055  min=1.028  max=1.206
pH lab            mean=5.897  std=0.143  min=5.750  max=6.140
Moisture content  mean=0.118  std=0.012  min=0.101  max=0.135

```

```

[7]: print(dfRand.columns.tolist())
      print(dfRand.head(3))

```

```

['Number', 'Treatment', 'Treatment?', 'Moist Weight (g)', 'Volume', 'Dry weight (g)', 'Moisture content', 'BD', 'pH lab']

```

```

Number Treatment Treatment? Moist Weight (g)      Volume Dry weight (g) \
0      R1    control         0          316.67  264.293047      282.92
1      R2    control         0          256.69  264.293047      223.07
2      R3    control         0          213.47  264.293047      189.69

```

```

Moisture content      BD pH lab
0          0.119292  1.070478  6.29
1          0.150715  0.844025  6.42
2          0.125362  0.717726  6.15

```

1.3.2 2b. LOI & MgO Baseline Stats by Group

LOI (Loss on Ignition) is a proxy for soil organic matter. MgO is the primary ERW tracer given the feedstock composition (47.3 wt% MgO). All values are LOI-normalized to remove organic matter dilution effects.

```

[8]: print(f'{"="*60}')
      print('LOI STATS (raw %)')
      print(f'{"="*60}')
      for name, df in GROUPS:
          loi = df['LOI']

```

```

print(f'\n{name} (n={len(df)}):')
print(f'  Mean={loi.mean():.2f}%  Std={loi.std():.2f}%  Min={loi.min():.
↳2f}%  Max={loi.max():.2f}%')

print(f'\n{"="*60}')
print('MgO LOI-NORMALIZED STATS (%)')
print(f'\n{"="*60}')
for name, df in GROUPS:
    mgo = df['MgO_norm']
    print(f'\n{name} (n={len(df)}):')
    print(f'  Mean={mgo.mean():.3f}%  Std={mgo.std():.3f}%  Min={mgo.min():.
↳3f}%  Max={mgo.max():.3f}%')

```

```

=====
LOI STATS (raw %)
=====

```

```

Treatment Paired (n=10):
  Mean=16.91%  Std=0.48%  Min=15.80%  Max=17.65%

```

```

Treatment Interior (n=7):
  Mean=15.48%  Std=0.22%  Min=15.15%  Max=15.80%

```

```

Control Paired (n=10):
  Mean=16.66%  Std=1.05%  Min=15.50%  Max=19.10%

```

```

Control Interior (n=5):
  Mean=16.51%  Std=0.70%  Min=15.50%  Max=17.30%

```

```

=====
MgO LOI-NORMALIZED STATS (%)
=====

```

```

Treatment Paired (n=10):
  Mean=2.199%  Std=0.046%  Min=2.123%  Max=2.272%

```

```

Treatment Interior (n=7):
  Mean=2.129%  Std=0.030%  Min=2.082%  Max=2.173%

```

```

Control Paired (n=10):
  Mean=2.172%  Std=0.051%  Min=2.088%  Max=2.235%

```

```

Control Interior (n=5):
  Mean=2.186%  Std=0.025%  Min=2.150%  Max=2.214%

```

```

[9]: # Export baseline summary table
rows = []

```



```

for name, df in GROUPS:
    loi = df['LOI']
    mgo = df['MgO_norm']
    rows.append({
        'Group': name, 'n': len(df),
        'LOI Mean (%)': round(loj.mean(), 2), 'LOI Std': round(loj.std(), 2),
        'LOI Min': round(loj.min(), 2), 'LOI Max': round(loj.max(), 2),
        'MgO_norm Mean (%)': round(mgo.mean(), 3), 'MgO_norm Std': round(mgo.
↪std(), 3),
        'MgO_norm Min': round(mgo.min(), 3), 'MgO_norm Max': round(mgo.max(),
↪3),
    })
summary = pd.DataFrame(rows)
summary.to_excel('outputs/baseline_summary_table.xlsx', index=False)
print('Saved: outputs/baseline_summary_table.xlsx')
display(summary)

```

Saved: outputs/baseline_summary_table.xlsx

	Group	n	LOI Mean (%)	LOI Std	LOI Min	LOI Max \
0	Treatment Paired	10	16.91	0.48	15.80	17.65
1	Treatment Interior	7	15.48	0.22	15.15	15.80
2	Control Paired	10	16.66	1.05	15.50	19.10
3	Control Interior	5	16.51	0.70	15.50	17.30

	MgO_norm Mean (%)	MgO_norm Std	MgO_norm Min	MgO_norm Max
0	2.199	0.046	2.123	2.272
1	2.129	0.030	2.082	2.173
2	2.172	0.051	2.088	2.235
3	2.186	0.025	2.150	2.214

```

[17]: # Baseline base cation pool by group
print("="*60)
print("BASELINE BASE CATION POOL (mmol+/kg) BY GROUP")
print("="*60)
for name, df in GROUPS:
    mmol = df.apply(lambda r: mmol_charge(r, 'norm'), axis=1)
    print(f"\n{name} (n={len(df)}):")
    print(f"  Mean={mmol.mean():.1f}  SD={mmol.std():.1f}  "
          f"Min={mmol.min():.1f}  Max={mmol.max():.1f}")

```

```

=====
BASELINE BASE CATION POOL (mmol+/kg) BY GROUP
=====

```

```

Treatment Paired (n=10):
  Mean=2346.9  SD=68.7  Min=2282.5  Max=2494.8

```

Treatment Interior (n=7):
Mean=2233.5 SD=51.8 Min=2153.7 Max=2289.7

Control Paired (n=10):
Mean=2314.3 SD=81.1 Min=2206.6 Max=2444.5

Control Interior (n=5):
Mean=2359.4 SD=67.8 Min=2241.5 Max=2404.7

1.4 3. Control Plot Representativeness — SMD Check

Rainbow Standard requires the Standardized Mean Difference (SMD) between treatment and control plots to be < 10% across key soil characteristics, confirming the control plot is representative of the treatment area at baseline.

SMD formula: $| \text{mean_treatment} - \text{mean_control} | / \text{pooled_std} \times 100$

```
[10]: def smd(g1, g2, col):
    m1, m2 = g1[col].mean(), g2[col].mean()
    s1, s2 = g1[col].std(), g2[col].std()
    n1, n2 = len(g1), len(g2)
    pooled = np.sqrt(((n1-1)*s1**2 + (n2-1)*s2**2) / (n1+n2-2))
    return abs(m1 - m2) / pooled * 100

print(f'{"="*58}')
print('SMD CHECK - Treatment Paired vs Control Paired')
print('Rainbow threshold: SMD < 10% required')
print(f'{"="*58}')

smd_checks = [
    ('MgO_norm', 'MgO (LOI-normalized)'),
    ('LOI', 'LOI (raw %)'),
    ('CaO_norm', 'CaO (LOI-normalized)'),
    ('K2O_norm', 'K2O (LOI-normalized)'),
    ('Na2O_norm', 'Na2O (LOI-normalized)'),
    ('SiO2_norm', 'SiO2 (LOI-normalized)'),
]

smd_results = []
for col, label in smd_checks:
    try:
        s = smd(treat_paired, ctrl_paired, col)
        status = 'PASS' if s < 10 else 'FAIL'
        smd_results.append({'Variable': label, 'SMD (%)': round(s,1), 'Status':
↪status})
        mark = 'OK' if s < 10 else '!!'
        print(f' {label:<28} SMD={s:5.1f}% {mark}')
```

```

except Exception as e:
    print(f' {label:<28} ERROR: {e}')

smd_df = pd.DataFrame(smd_results)
smd_df.to_excel('outputs/smd_check.xlsx', index=False)
print('\nSaved: outputs/smd_check.xlsx')

```

```

=====
SMD CHECK - Treatment Paired vs Control Paired
Rainbow threshold: SMD < 10% required
=====

```

MgO (LOI-normalized)	SMD= 56.8%	!!
LOI (raw %)	SMD= 30.6%	!!
CaO (LOI-normalized)	SMD= 29.0%	!!
K2O (LOI-normalized)	SMD= 26.2%	!!
Na2O (LOI-normalized)	SMD= 48.8%	!!
SiO2 (LOI-normalized)	SMD= 5.1%	OK

Saved: outputs/smd_check.xlsx

```

[11]: import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter

smd_full_rows = []
for col, label in smd_checks:
    t_mean = treat_paired[col].mean()
    c_mean = ctrl_paired[col].mean()
    abs_diff = abs(t_mean - c_mean)
    t_std = treat_paired[col].std()
    c_std = ctrl_paired[col].std()
    n1, n2 = len(treat_paired), len(ctrl_paired)
    pooled = np.sqrt(((n1-1)*t_std**2 + (n2-1)*c_std**2) / (n1+n2-2))
    smd_val = (abs_diff / pooled) * 100
    smd_full_rows.append({
        'Variable': label, 'Treatment Mean': round(t_mean,4), 'Control Mean':
        ↪round(c_mean,4),
        'Abs Difference': round(abs_diff,4), 'Pooled SD': round(pooled,4),
        'SMD (%)': round(smd_val,1), 'Status': 'PASS' if smd_val < 10 else
        ↪'FAIL',
    })
smd_full = pd.DataFrame(smd_full_rows)

wb = openpyxl.Workbook()
ws = wb.active
ws.title = 'SMD Context Table'
hfont = Font(name='Arial', bold=True, color='FFFFFF', size=11)

```

```

hfill = PatternFill('solid', start_color='1a5a8a')
pfill = PatternFill('solid', start_color='C6EFCE')
ffill = PatternFill('solid', start_color='FFCCCC')
ctr    = Alignment(horizontal='center')
thin   = Side(style='thin', color='CCCCCC')
bord   = Border(left=thin, right=thin, top=thin, bottom=thin)

headers = list(smd_full.columns)
ws.append(headers)
for ci, _ in enumerate(headers, 1):
    cell = ws.cell(1, ci)
    cell.font=hfont; cell.fill=hfill; cell.alignment=ctr; cell.border=bord

for row in smd_full.itertuples(index=False):
    ws.append(list(row))
    r = ws.max_row
    fill = pfill if ws.cell(r, headers.index('Status')+1).value=='PASS' else ffill
    for c in range(1, len(headers)+1):
        cell = ws.cell(r, c)
        cell.fill=fill; cell.alignment=ctr; cell.border=bord; cell.
        font=Font(name='Arial',size=10)

ws.append([])
nr = ws.max_row + 1
ws.cell(nr, 1, 'Note:').font = Font(name='Arial', bold=True, size=10)
ws.cell(nr+1, 1, 'FAIL results are driven by unusually low within-group
variability '
        '(pooled SD < 0.05%), not by meaningful differences between zones. '
        'Absolute differences are agronomically negligible. '
        'Both zones are geologically homogeneous at baseline.').font =
Font(name='Arial', italic=True, size=10)
ws.merge_cells(f'A{nr+1}:{get_column_letter(len(headers))}{nr+1}')
ws.cell(nr+1,1).alignment = Alignment(wrap_text=True)
ws.row_dimensions[nr+1].height = 45
for i, w in enumerate([24,16,14,15,12,10,10], 1):
    ws.column_dimensions[get_column_letter(i)].width = w
wb.save('outputs/smd_context_table.xlsx')
print('Saved: outputs/smd_context_table.xlsx')
display(smd_full)

```

Saved: outputs/smd_context_table.xlsx

	Variable	Treatment Mean	Control Mean	Abs Difference \
0	MgO (LOI-normalized)	2.1990	2.1715	0.0274
1	LOI (raw %)	16.9100	16.6600	0.2500
2	CaO (LOI-normalized)	1.5867	1.5414	0.0453
3	K2O (LOI-normalized)	1.4647	1.4786	0.0139

4	Na2O (LOI-normalized)	1.1736	1.1559	0.0177
5	SiO2 (LOI-normalized)	58.2740	58.2960	0.0220

	Pooled SD	SMD (%)	Status
0	0.0483	56.8	FAIL
1	0.8164	30.6	FAIL
2	0.1563	29.0	FAIL
3	0.0530	26.2	FAIL
4	0.0362	48.8	FAIL
5	0.4295	5.1	PASS

1.5 4. Pre vs Post Application Analysis

Paired T-tests comparing the same sample locations before and after olivine application. All 17 treatment samples (10 paired zone + 7 interior) are matched to their post-application equivalents.

Primary tracer: LOI-normalized MgO — chosen because the TS-1 olivine feedstock is 47.3 wt% MgO, providing a high signal-to-noise ratio against the baseline soil (~2.1-2.2% MgO normalized).

```
[12]: # Build paired pre/post dataframe
pre_treat = dfpre[dfpre['pair_key'].notna()].copy()
post_treat = dfpost[dfpost['pair_key'].notna()].copy()

merge_cols = ['pair_key', 'SAMPLE'] + BASE_OXIDES + [c+'_norm' for c in BASE_OXIDES]

paired = pd.merge(
    pre_treat[merge_cols],
    post_treat[merge_cols],
    on='pair_key', how='inner',
    suffixes=('_pre', '_post')
).sort_values('pair_key').reset_index(drop=True)

for ox in BASE_OXIDES:
    for sfx in ('_pre', '_post'):
        paired[f'{ox}{sfx}'] = pd.to_numeric(paired[f'{ox}{sfx}'],
        errors='coerce')
        paired[f'{ox}_norm{sfx}'] = pd.to_numeric(paired[f'{ox}_norm{sfx}'],
        errors='coerce')

paired['mmol_pre'] = paired.apply(lambda r: mmol_charge(r, 'pre'), axis=1)
paired['mmol_post'] = paired.apply(lambda r: mmol_charge(r, 'post'), axis=1)
paired['delta_mmol'] = paired['mmol_post'] - paired['mmol_pre']
paired['BaseCat_PctChange'] = (paired['delta_mmol'] / paired['mmol_pre']) * 100
```

```

paired['BaseCat_Ratio_PostPre'] = paired['mmol_post'] / paired['mmol_pre']

print(f'Paired samples: {len(paired)}')
print(f'Pre keys missing in post:  {sorted(set(pre_treat["pair_key"]) -
↪set(post_treat["pair_key"])))}')
print(f'Post keys missing in pre:  {sorted(set(post_treat["pair_key"]) -
↪set(pre_treat["pair_key"])))}')
print('Paired dataframe ready')

```

Paired samples: 17

Pre keys missing in post: ['CR01', 'CR02', 'CR03', 'CR04', 'CR05', 'CZ01',
'CZ02', 'CZ03', 'CZ04', 'CZ05', 'CZ06', 'CZ07', 'CZ08', 'CZ09', 'CZ10']

Post keys missing in pre: []

Paired dataframe ready

1.6 4. HL Notes Re Pre vs Post Application Analysis

1.6.1 Control Plot Reference — Design Note

Rather than incurring additional fees for sampling lab analysis on the control plot post application, we decided to use the pre treatment control data. Control did not receive olvine, it will remain geochemically stable over the monitoring period. Pre-application control measurements (ALS Report LR25329328) therefore serve as the post-application control reference for this analysis, consistent with Rainbow Method 2 solid-phase mass balance design.

This assumption is supported by four lines of evidence: 1. No agronomic inputs (fertilizer, lime, seed) were applied to the control plot before or after olvine application to the treatment area. 2. Soil inorganic carbon (SIC) was below laboratory detection limits at baseline, confirming no carbonate kinetics that could alter base cation concentrations over short timescales. 3. Our control data shows flat cumulative TA efflux following the September 2025 application event, while the treatment group diverges — confirming control geochemical stability post-application (see Cascade Signal-to-Noise Report). 4. The short monitoring window (baseline: May–July 2025; post-application sampling: September 2025) limits the possibility of natural geochemical drift in the undisturbed control.

Where statistical significance is shown, we are not claiming anything beyond that.

```

[13]: # Paired T-tests
print(f'{"="*58}')
print('PAIRED T-TEST RESULTS (n=17)')
print(f'{"="*58}')

mgo_pre_n = pd.to_numeric(paired['MgO_norm_pre'], errors='coerce')
mgo_post_n = pd.to_numeric(paired['MgO_norm_post'], errors='coerce')
mask_mgo = mgo_pre_n.notna() & mgo_post_n.notna()
tstat_mgo, pval_mgo = stats.ttest_rel(mgo_post_n[mask_mgo], mgo_pre_n[mask_mgo])
diff_mgo = mgo_post_n[mask_mgo] - mgo_pre_n[mask_mgo]
d_mgo = diff_mgo.mean() / diff_mgo.std(ddof=1)

```

```

print(f'\n1. LOI-Normalized MgO (primary tracer):')
print(f'   Mean delta MgO_norm = {diff_mgo.mean():.4f}%')
print(f'   t = {tstat_mgo:.3f}')
print(f'   p = {pval_mgo:.2e}')
print(f'   Cohens d = {d_mgo:.3f}')
print(f'   Result: {"SIGNIFICANT" if pval_mgo < 0.05 else "NOT SIGNIFICANT"}')

mgo_pre_r = pd.to_numeric(paired['MgO_pre'], errors='coerce')
mgo_post_r = pd.to_numeric(paired['MgO_post'], errors='coerce')
mask_r = mgo_pre_r.notna() & mgo_post_r.notna()
tstat_r, pval_r = stats.ttest_rel(mgo_post_r[mask_r], mgo_pre_r[mask_r])
diff_r = mgo_post_r[mask_r] - mgo_pre_r[mask_r]
d_r = diff_r.mean() / diff_r.std(ddof=1)

print(f'\n2. Raw MgO (unnormalized, for reference):')
print(f'   Mean delta MgO = {diff_r.mean():.4f}%')
print(f'   t = {tstat_r:.3f}')
print(f'   p = {pval_r:.2e}')
print(f'   Cohens d = {d_r:.3f}')

tstat_bc, pval_bc = stats.ttest_rel(paired['mmol_post'], paired['mmol_pre'])
diff_bc = paired['mmol_post'] - paired['mmol_pre']
d_bc = diff_bc.mean() / diff_bc.std(ddof=1)

print(f'\n3. Base Cation Pool (CaO+MgO+Na2O+K2O as mmol+/kg):')
print(f'   Mean delta mmol = {diff_bc.mean():.1f} mmol+/kg')
print(f'   t = {tstat_bc:.3f}')
print(f'   p = {pval_bc:.2e}')
print(f'   Cohens d = {d_bc:.3f}')
print(f'   Result: {"SIGNIFICANT" if pval_bc < 0.05 else "NOT SIGNIFICANT"}')

```

=====

PAIRED T-TEST RESULTS (n=17)

=====

1. LOI-Normalized MgO (primary tracer):
 - Mean delta MgO_norm = 0.4629%
 - t = 11.350
 - p = 4.59e-09
 - Cohens d = 2.753
 - Result: SIGNIFICANT
2. Raw MgO (unnormalized, for reference):
 - Mean delta MgO = 0.2882%
 - t = 9.818
 - p = 3.55e-08
 - Cohens d = 2.381

3. Base Cation Pool (CaO+MgO+Na2O+K2O as mmol+/kg):

Mean delta mmol = 83.4 mmol+/kg

t = 5.006

p = 1.29e-04

Cohens d = 1.214

Result: SIGNIFICANT

1.6.2 Charts: Pre vs Post Comparison

```
[14]: fig, axes = plt.subplots(2, 2, figsize=(13, 11))
fig.suptitle(
    'Mallonee Family Farm - Tilled ERW Project\nPre vs Post Olivine Application_
    ↳Analysis',
    fontsize=13, fontweight='bold', y=1.01
)

ax = axes[0,0]
x1 = paired['MgO_norm_pre'].values
y1 = paired['MgO_norm_post'].values
labels1 = paired['pair_key'].values
m1 = ~np.isnan(x1) & ~np.isnan(y1)
ax.scatter(x1[m1], y1[m1], color='#1a5a8a', zorder=3, s=60)
mn, mx = min(x1[m1].min(), y1[m1].min()), max(x1[m1].max(), y1[m1].max())
ax.plot([mn,mx],[mn,mx], '--', color='gray', linewidth=1, label='1:1 line')
for xi, yi, lab in zip(x1[m1], y1[m1], labels1[m1]):
    ax.annotate(lab, (xi,yi), fontsize=7, xytext=(4,4), textcoords='offset_
    ↳points')
ax.set_xlabel('Pre LOI-Normalized MgO (%)')
ax.set_ylabel('Post LOI-Normalized MgO (%)')
ax.set_title(f'LOI-Normalized MgO: Pre vs Post\np={pval_mgo:.2e}, d={d_mgo:.
    ↳2f}')
ax.legend(fontsize=8); ax.grid(alpha=0.3)

ax = axes[0,1]
delta_mgo_norm = (paired['MgO_norm_post'] - paired['MgO_norm_pre']).dropna()
ax.boxplot(delta_mgo_norm, labels=['delta MgO_norm'], patch_artist=True,
    boxprops=dict(facecolor='#cce0f0'))
ax.axhline(0, color='gray', linestyle='--', linewidth=1)
ax.set_ylabel('Post - Pre LOI-Normalized MgO (%)')
ax.set_title(f'Change in LOI-Normalized MgO\nMean={delta_mgo_norm.mean():.3f}%')
ax.grid(alpha=0.3)

ax = axes[1,0]
x3 = paired['mmol_pre'].values
y3 = paired['mmol_post'].values
labels3 = paired['pair_key'].values
```



```

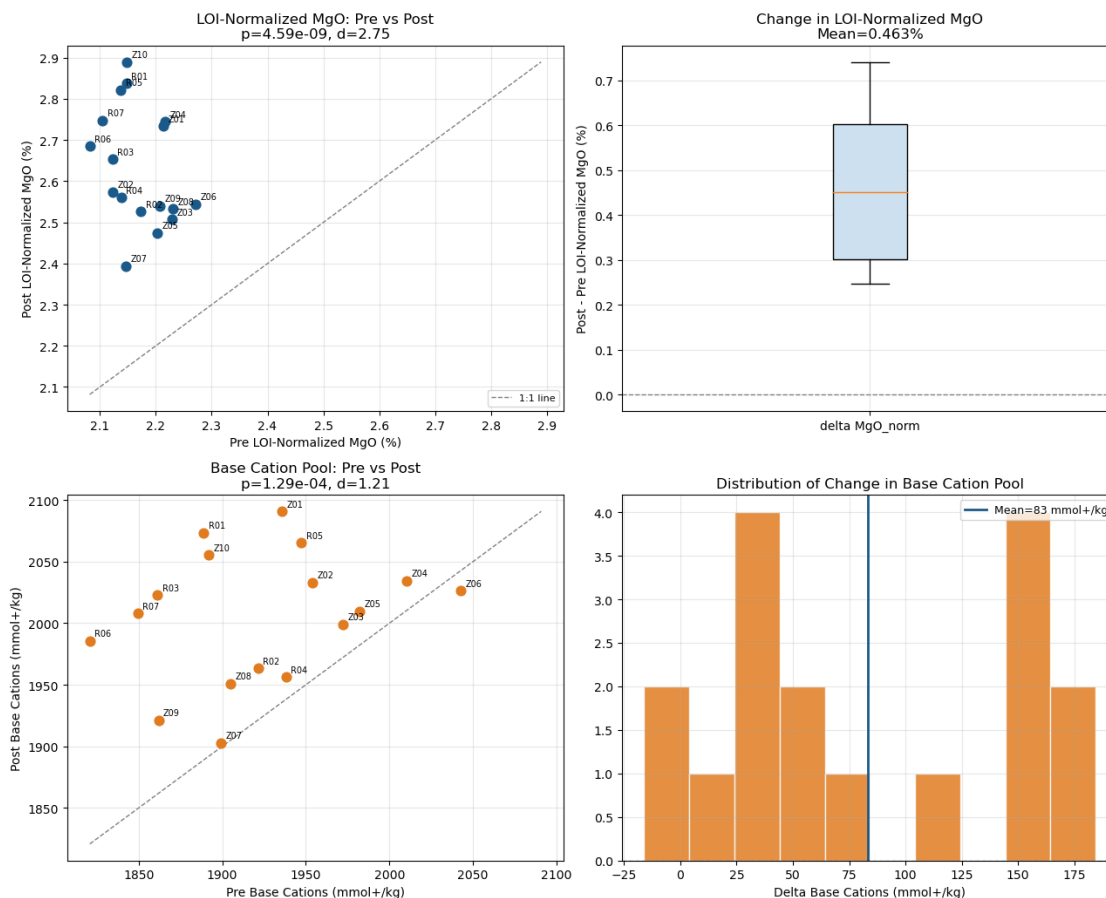
m3 = ~np.isnan(x3) & ~np.isnan(y3)
ax.scatter(x3[m3], y3[m3], color='#e07b20', zorder=3, s=60)
mn3, mx3 = min(x3[m3].min(), y3[m3].min()), max(x3[m3].max(), y3[m3].max())
ax.plot([mn3,mx3],[mn3,mx3], '--', color='gray', linewidth=1)
for xi, yi, lab in zip(x3[m3], y3[m3], labels3[m3]):
    ax.annotate(lab, (xi,yi), fontsize=7, xytext=(4,4), textcoords='offset_
↳points')
ax.set_xlabel('Pre Base Cations (mmol+/kg)')
ax.set_ylabel('Post Base Cations (mmol+/kg)')
ax.set_title(f'Base Cation Pool: Pre vs Post\np={pval_bc:.2e}, d={d_bc:.2f}')
ax.grid(alpha=0.3)

ax = axes[1,1]
delta_bc = paired['delta_mmol'].dropna()
ax.hist(delta_bc, bins=10, color='#e07b20', edgecolor='white', alpha=0.85)
ax.axhline(0, color='gray', linestyle='--', linewidth=1)
ax.axvline(delta_bc.mean(), color='#1a5a8a', linestyle='-',
            linewidth=2, label=f'Mean={delta_bc.mean():.0f} mmol+/kg')
ax.set_xlabel('Delta Base Cations (mmol+/kg)')
ax.set_title('Distribution of Change in Base Cation Pool')
ax.legend(fontsize=9); ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('outputs/mallonee_pre_post_charts.png', dpi=150,
↳bbox_inches='tight')
plt.show()
print('Saved: outputs/mallonee_pre_post_charts.png')

```

Mallonee Family Farm - Tilled ERW Project Pre vs Post Olivine Application Analysis



Saved: outputs/mallonee_pre_post_charts.png

1.7 Section 4b: Post-Application Treatment vs Baseline Control

1.7.1 Control Plot Reference — Design Note

Post-application control sampling was not conducted as the control area received no olivine treatment and is expected to remain geochemically stable. Pre-application control measurements (ALS Report LR25329328) serve as the post-application control reference, consistent with Rainbow Method 2 solid-phase mass balance design.

This assumption is supported by four lines of evidence:

1. No agronomic inputs (fertilizer, lime, seed) were applied to the control plot before or after olivine application to the treatment area.
2. Soil inorganic carbon (SIC) was below laboratory detection limits at baseline, confirming no carbonate kinetics that could alter base cation concentrations over short timescales.
3. Everest sensor data shows flat cumulative TA efflux in the control group following the September 2025 application event, while the treatment group diverges — confirming control geochem-

ical stability post-application (see Cascade Signal-to-Noise Report).

4. The short monitoring window (baseline: May–July 2025; post-application sampling: September 2025) limits the possibility of natural geochemical drift in the undisturbed control.

Where statistical significance cannot be demonstrated, no CDR claims are made.

```
[15]: OXIDE_COLS_NORM = [c+'_norm' for c in BASE_OXIDES]

# Isolate post-application treatment groups
# bug fix - matches actual ALS codes in post-application file
post_zones = dfpost[dfpost['SAMPLE'].str.match(r'TZ\s+\d+', case=False,
↳na=False)].copy()
post_rand = dfpost[dfpost['SAMPLE'].str.match(r'TT rand', case=False,
↳na=False)].copy()

# Comparison 1: all post-treatment vs all baseline control (paired zones)
t1, p1 = stats.ttest_ind(
    dfpost['MgO_norm'].dropna(),
    ctrl_paired['MgO_norm'].dropna()
)
print("="*58)
print("POST-TREATMENT vs BASELINE CONTROL - MgO_norm")
print("="*58)
print(f"\n1. All post-treatment (n={len(dfpost)}) vs baseline control paired_
↳(n={len(ctrl_paired)})")
print(f"    Post-treatment mean:    {dfpost['MgO_norm'].mean():.3f}%  "
↳SD={dfpost['MgO_norm'].std():.3f}")
print(f"    Baseline control mean:    {ctrl_paired['MgO_norm'].mean():.3f}%  "
↳SD={ctrl_paired['MgO_norm'].std():.3f}")
print(f"    t={t1:.3f}, p={p1:.2e}")
print(f"    Result: {'SIGNIFICANT (p<0.05)' if p1<0.05 else 'NOT SIGNIFICANT'}")

# Comparison 2: post-treatment zones vs baseline control zones (matched_
↳locations)
t2, p2 = stats.ttest_ind(
    post_zones['MgO_norm'].dropna(),
    ctrl_paired['MgO_norm'].dropna()
)
print(f"\n2. Post-treatment zones only (n={len(post_zones)}) vs baseline_
↳control zones (n={len(ctrl_paired)})")
print(f"    Post-treatment zones mean:    {post_zones['MgO_norm'].mean():.3f}%")
print(f"    Baseline control mean:        {ctrl_paired['MgO_norm'].mean():.3f}%")
print(f"    t={t2:.3f}, p={p2:.2e}")
print(f"    Result: {'SIGNIFICANT (p<0.05)' if p2<0.05 else 'NOT SIGNIFICANT'}")

# Note on Z06 and BD anomaly
print("""
```

Notes:

- Z06 shows a small negative delta base cation value (-16 mmol+/kg, -0.79%) in the paired audit table. MgO_norm delta for Z06 remains positive (+0.215%). This is within analytical noise and does not affect the overall result.
 - Treatment interior random samples (M-R-D) show higher bulk density (mean 1.132 g/cm³) than paired transect zones (mean 0.789 g/cm³). This is consistent with their lower mean LOI (15.48% vs 16.91%), reflecting lower organic matter content in the western field interior, consistent with the documented west-to-east LOI gradient. This is not a measurement error.
 - Post-application control pH, BD and CEC not yet collected. Planned for September 2026 monitoring round.
- """)

=====

POST-TREATMENT vs BASELINE CONTROL - MgO_norm

=====

1. All post-treatment (n=17) vs baseline control paired (n=10)
Post-treatment mean: 2.633% SD=0.143
Baseline control mean: 2.172% SD=0.051
t=9.785, p=4.96e-10
Result: SIGNIFICANT (p<0.05)
2. Post-treatment zones only (n=10) vs baseline control zones (n=10)
Post-treatment zones mean: 2.593%
Baseline control mean: 2.172%
t=8.450, p=1.12e-07
Result: SIGNIFICANT (p<0.05)

Notes:

- Z06 shows a small negative delta base cation value (-16 mmol+/kg, -0.79%) in the paired audit table. MgO_norm delta for Z06 remains positive (+0.215%). This is within analytical noise and does not affect the overall result.
- Treatment interior random samples (M-R-D) show higher bulk density (mean 1.132 g/cm³) than paired transect zones (mean 0.789 g/cm³). This is consistent with their lower mean LOI (15.48% vs 16.91%), reflecting lower organic matter content in the western field interior, consistent with the documented west-to-east LOI gradient. This is not a measurement error.
- Post-application control pH, BD and CEC not yet collected. Planned for September 2026 monitoring round.

1.8 5. Audit Table

Full paired sample results exported for submission with the site characterization report.

```
[16]: audit = paired[[
    'pair_key', 'SAMPLE_pre', 'SAMPLE_post',
    'MgO_pre', 'MgO_post',
    'MgO_norm_pre', 'MgO_norm_post',
    'mmol_pre', 'mmol_post', 'delta_mmol',
    'BaseCat_PctChange', 'BaseCat_Ratio_PostPre'
]].copy()

audit.columns = [
    'Pair_Key', 'Pre_Sample_ID', 'Post_Sample_ID',
    'MgO_Pre_raw_%', 'MgO_Post_raw_%',
    'MgO_Pre_norm_%', 'MgO_Post_norm_%',
    'BaseCat_Pre_mmolkg', 'BaseCat_Post_mmolkg',
    'Delta_BaseCat_mmolkg', 'BaseCat_PctChange', 'BaseCat_Ratio_PostPre'
]

audit = audit.round(4)
audit.to_csv('outputs/mallonee_tilled_audit_pairs.csv', index=False)
audit.to_excel('outputs/mallonee_tilled_audit_pairs.xlsx', index=False)
print('Saved: outputs/mallonee_tilled_audit_pairs.csv')
print('Saved: outputs/mallonee_tilled_audit_pairs.xlsx')
display(audit)
```

Saved: outputs/mallonee_tilled_audit_pairs.csv

Saved: outputs/mallonee_tilled_audit_pairs.xlsx

	Pair_Key	Pre_Sample_ID	Post_Sample_ID	MgO_Pre_raw_%	MgO_Post_raw_%	\
0	R01	Treatment_1	TT rand 1	1.82	2.21	
1	R02	Treatment_2	TT rand 2	1.84	2.01	
2	R03	Treatment_3	TT rand 3	1.78	2.10	
3	R04	Treatment_4	TT rand 4	1.81	2.05	
4	R05	Treatment_5	TT rand 5	1.82	2.26	
5	R06	Treatment_6	TT rand 6	1.76	2.17	
6	R07	Treatment_7	TT rand 7	1.78	2.17	
7	Z01	Treatment_z1	TZ 1	1.86	2.21	
8	Z02	Treatment_z2	TZ 2	1.76	2.08	
9	Z03	Treatment_z3	TZ 3	1.88	2.03	
10	Z04	Treatment_z4	TZ 4	1.88	2.16	
11	Z05	Treatment_z5	TZ 5	1.80	1.98	
12	Z06	Treatment_z6	TZ 6	1.86	2.02	
13	Z07	Treatment_z7	TZ 7	1.76	1.92	
14	Z08	Treatment_z8	TZ 8	1.85	2.00	
15	Z09	Treatment_z9	TZ 9	1.80	2.04	

16	Z10	Treatment_z10	TZ 10	1.78	2.33
	MgO_Pre_norm_%	MgO_Post_norm_%	BaseCat_Pre_mmolkg	BaseCat_Post_mmolkg	\
0	2.1472	2.8377	1888.8638	2073.3220	
1	2.1734	2.5261	1921.3753	1963.3684	
2	2.1223	2.6535	1861.1168	2022.6382	
3	2.1382	2.5596	1938.2445	1956.2163	
4	2.1377	2.8211	1947.1974	2065.1923	
5	2.0819	2.6846	1820.7040	1985.7105	
6	2.1050	2.7479	1849.4816	2008.0447	
7	2.2135	2.7341	1935.6315	2090.8161	
8	2.1230	2.5733	1953.7640	2033.0932	
9	2.2288	2.5071	1972.2207	1998.9392	
10	2.2172	2.7446	2010.4300	2034.5822	
11	2.2032	2.4741	1982.1067	2009.7886	
12	2.2719	2.5447	2042.4529	2026.4119	
13	2.1469	2.3943	1899.0810	1902.9106	
14	2.2303	2.5323	1904.8559	1951.1034	
15	2.2067	2.5398	1861.8669	1921.0249	
16	2.1482	2.8897	1891.6829	2055.4345	
	Delta_BaseCat_mmolkg	BaseCat_PctChange	BaseCat_Ratio_PostPre		
0	184.4582	9.7656	1.0977		
1	41.9930	2.1856	1.0219		
2	161.5214	8.6787	1.0868		
3	17.9718	0.9272	1.0093		
4	117.9950	6.0597	1.0606		
5	165.0065	9.0628	1.0906		
6	158.5630	8.5734	1.0857		
7	155.1847	8.0173	1.0802		
8	79.3292	4.0603	1.0406		
9	26.7185	1.3547	1.0135		
10	24.1523	1.2013	1.0120		
11	27.6819	1.3966	1.0140		
12	-16.0410	-0.7854	0.9921		
13	3.8296	0.2017	1.0020		
14	46.2476	2.4279	1.0243		
15	59.1580	3.1773	1.0318		
16	163.7515	8.6564	1.0866		

1.9 6. Summary of Key Findings

Statistical results for inclusion in the Site Characterization Report.

```
[16]: print('=' * 60)
      print('SUMMARY OF KEY FINDINGS')
```

```

print('Mallonee Family Farm - Tilled ERW Project')
print('ALS Reports: LR25329328 (baseline) | RE25329687 (post)')
print('=' * 60)

treat_ph = dfpH[dfpH['Treatment'] == 'Baseline (Pre-Application)']
print('\nBASELINE SOIL CONDITIONS (Treatment Paired Zones, n=10)')
print(f'  pH (lab):          mean={treat_ph["pH lab"].mean():.2f}')
print(f'  Bulk density:       mean={treat_ph["BD"].mean():.3f} g/cm3')
print(f'  LOI:                mean={treat_ph["LOI"].mean():.2f}% '
      f'(range {treat_ph["LOI"].min():.2f}-{treat_ph["LOI"].max():.2f})')
print(f'  MgO (normalized): mean={treat_ph["MgO_norm"].mean():.3f}%')
print('\nRANDOM INTERIOR SAMPLES (Treatment n=7, Control n=5)')
print(f'  Loaded from: {PATH_RAND}')
print(f'  BD and pH stats reported in Section 2a above')
print('\nCONTROL PLOT REPRESENTATIVENESS')
print('  See outputs/smd_context_table.xlsx')
print('\nPRE vs POST APPLICATION (n=17 paired samples)')
print(f'  LOI-normalized MgO:')
print(f'    Mean delta = {diff_mgo.mean():.4f}%')
print(f'    t={tstat_mgo:.3f}  p={pval_mgo:.2e}  d={d_mgo:.3f}')
print(f'    Result: {"SIGNIFICANT (p<0.05)" if pval_mgo < 0.05 else "NOT_'}
      f'SIGNIFICANT"')
print(f'  Base Cation Pool:')
print(f'    Mean delta = {diff_bc.mean():.1f} mmol+/kg')
print(f'    t={tstat_bc:.3f}  p={pval_bc:.2e}  d={d_bc:.3f}')
print(f'    Result: {"SIGNIFICANT (p<0.05)" if pval_bc < 0.05 else "NOT_'}
      f'SIGNIFICANT"')
print('\nOUTPUT FILES')
for f in ['baseline_summary_table.xlsx', 'smd_check.xlsx', 'smd_context_table.
      f'xlsx',
          'mallonee_tilled_audit_pairs.csv', 'mallonee_tilled_audit_pairs.xlsx',
          'mallonee_pre_post_charts.png']:
    print(f'  outputs/{f}')

```

SUMMARY OF KEY FINDINGS

Mallonee Family Farm - Tilled ERW Project
 ALS Reports: LR25329328 (baseline) | RE25329687 (post)

BASELINE SOIL CONDITIONS (Treatment Paired Zones, n=10)

pH (lab): mean=6.11
 Bulk density: mean=0.789 g/cm3
 LOI: mean=16.91% (range 15.80-17.65%)
 MgO (normalized): mean=2.199%

RANDOM INTERIOR SAMPLES (Treatment n=7, Control n=5)

Loaded from: csv files/Random Interior Samples tilled pre-treatment and control Mallonee - Sheet1 (1).csv

BD and pH stats reported in Section 2a above

CONTROL PLOT REPRESENTATIVENESS

See outputs/smd_context_table.xlsx

PRE vs POST APPLICATION (n=17 paired samples)

LOI-normalized MgO:

Mean delta = 0.4629%

t=11.350 p=4.59e-09 d=2.753

Result: SIGNIFICANT (p<0.05)

Base Cation Pool:

Mean delta = 83.4 mmol+/kg

t=5.006 p=1.29e-04 d=1.214

Result: SIGNIFICANT (p<0.05)

OUTPUT FILES

outputs/baseline_summary_table.xlsx

outputs/smd_check.xlsx

outputs/smd_context_table.xlsx

outputs/mallonee_tilled_audit_pairs.csv

outputs/mallonee_tilled_audit_pairs.xlsx

outputs/mallonee_pre_post_charts.png

```
[17]: import subprocess
subprocess.run(['jupyter', 'nbconvert', '--to', 'pdf',
               ↪ 'Mallonee_Tilled_ERW_Analysis_v3.ipynb'])
```

```
[NbConvertApp] Converting notebook Mallonee_Tilled_ERW_Analysis_v3.ipynb to pdf
[NbConvertApp] Support files will be in Mallonee_Tilled_ERW_Analysis_v3_files/
[NbConvertApp] Making directory ./Mallonee_Tilled_ERW_Analysis_v3_files
[NbConvertApp] Writing 140123 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 249407 bytes to Mallonee_Tilled_ERW_Analysis_v3.pdf
```

```
[17]: CompletedProcess(args=['jupyter', 'nbconvert', '--to', 'pdf',
                             'Mallonee_Tilled_ERW_Analysis_v3.ipynb'], returncode=0)
```

```
[ ]:
```


[]:

[]:

[]: